SWOT Pipeline — Full Guide

This system turns a company's recent news and financial statements into an evidence-based SWOT draft that a human reviews and approves.

The pipeline has two sides:

- **News side:** Google News -> triage -> resolve links -> verify pages -> sentiment scores -> SWOT evidence
- **Financial side:** balance sheet CSV + income statement CSV -> mapping approval -> approved metrics -> SWOT evidence
- **Final step:** both sides feed `swot_builder.py`, which makes proposals; you review and approve them

Everything is saved with timestamps and hashes, so you can always trace which input produced which output.

Folder structure

```
vultr/
├── app.py
├── planning.ipynb
├── summaries.txt
├── assets/                  <- raw financial CSVs (input, never edited)
├── information_new/         <- news pipeline + SWOT builder (main working folder)
│   ├── company_profile.py
│   ├── discovery.py
│   ├── triage.py
│   ├── resolve.py
│   ├── verify.py
│   ├── sentiment.py
│   ├── fin_metrics.py
│   ├── swot_builder.py
│   ├── swot.py              <- older version, superseded by swot_builder.py
│   └── output/              <- all news + SWOT outputs land here
├── planning/                <- financial gate + planning analysis
│   ├── financial_gate.py
│   ├── run_planning.py
│   ├── planning.py
│   ├── label_mapper.py
│   ├── llm_client.py
│   ├── check_labels.py
│   ├── .env
│   └── output/              <- financial gate outputs land here
├── results/                 <- old planning analysis outputs (legacy)
├── testing/                 <- scratch/test scripts
└── clean_gm.py              <- one-off script that cleans GM CSVs
```

File-by-file explanation

assets/ — raw financial data (input only)

File	What it is
`*_balance_sheet_annual_2020_2026.csv`	Balance sheet, one row per line item, one column per year-end
`*_income_statement_annual_2020_2026.csv`	Income statement, same layout
`*_cleaned.csv`	Versions with problematic rows removed (debt, interest) before the gate runs

These are inputs. Never edit them by hand; use `clean_gm.py` (or equivalent) to produce `_cleaned` versions.

information_new/ — news pipeline + SWOT

File	What it does
`company_profile.py`	Defines company name, ticker, aliases, scope, exclusions
`discovery.py`	Stage 1: pulls Google News RSS for the last N days, filters by company, deduplicates
`triage.py`	Stage 2: clusters articles into events, labels them (topic/sentiment/signal) via LLM or keyword fallback, selects top 40
`resolve.py`	Stage 3: turns Google News redirect links into real publisher URLs
`verify.py`	Stage 4: opens each URL in a headless browser, extracts text, screenshots, checks title match
`sentiment.py`	Stage 5: scores each verified article's sentiment with the LLM, aggregates per company
`fin_metrics.py`	Computes financial metrics (margins, growth, ratios) from the approved dataset
`swot_builder.py`	Final stage: combines news + financial evidence, asks LLM to propose SWOT quadrant per item, writes draft
`swot.py`	Older/simpler version of the builder, kept for reference

planning/ — financial gate

File	What it does
`financial_gate.py`	Reads the two CSVs, maps rows to standard fields, checks accounting identities, asks you to approve, writes approved dataset
`run_planning.py`	Older script: maps rows via LLM and runs `planning.analyze` (legacy, superseded by the gate)
`planning.py`	Core analysis logic used by run_planning.py

File	What it does
label_mapper.py	Maps raw CSV row names to standard field names
llm_client.py	Shared LLM call helper (reads GROQ_API_KEY)
check_labels.py	Utility to inspect mapping results
.env	Environment variables (keys, model names)

Root-level scripts

File	What it does
clean_gm.py	Removes problematic rows (debt, interest) from GM CSVs before the gate runs
app.py	Not part of this pipeline
planning.ipynb	Notebook version of planning analysis
summaries.txt	Notes/scratch

Full pipeline: step by step

Step 0 — Set environment variables

Open PowerShell and run these once per session:

```
cd C:\Users\adeel\Desktop\notebooks\vultr\information_new
$env:VULTR_INFERENCE_BASE    = "https://api.groq.com/openai/v1"
$env:VULTR_INFERENCE_API_KEY = "gsk_your_key"
$env:VULTR_CHAT_MODEL        = "openai/gpt-oss-120b"
$env:SWOT_REASONING_EFFORT   = "low"
$env:SWOT_DELAY              = "12"
$env:GROQ_API_KEY            = $env:VULTR_INFERENCE_API_KEY
```

Check the key works:

```
python triage.py --list-models
```

If it prints model names, you're good.

Step 1 — Discovery (news collection)

```
python discovery.py --company "General Motors" --ticker GM --alias GM --exclude
"genetically modified" --location global
```

What it does: Pulls Google News RSS for the last 30 days, filters to articles mentioning the company, deduplicates.

Output:

```
output\<run_id>\
└── discovery_<run_id>.json
```

Contains: every unique article (title, source, URL, publish date, snippet, dedup info).

Step 2 — Triage (cluster + select)

```
python triage.py
```

(Uses the newest discovery run by default.)

What it does: Groups similar articles into "events", labels each with topic/sentiment/signal (LLM, falls back to keywords if API fails), scores and picks the top 40.

Output:

```
output\triage_<run_id>\
├── triage_<company>.json      <- all events with labels
├── selected_for_resolution.json <- top 40 events to resolve
└── label_cache.json            <- LLM label cache (so re-runs are cheap)
```

Step 3 — Resolve (get real URLs)

```
python resolve.py
```

What it does: Turns Google News redirect links into actual publisher URLs.

Output:

```
output\triage_<run_id>\
├── resolved.json              <- real URLs per event
└── resolved_failures.json     <- any that failed
```

Step 4 — Verify (fetch and check pages)

```
python verify.py
```

What it does: Opens each resolved URL in headless Chromium, extracts article text, takes a screenshot, checks the headline matches the page.

Output:

```
output\verify_triage_<run_id>\
├── articles\<event_id>.json    <- extracted text + metadata per article
├── raw\<event_id>.html         <- raw HTML
├── screenshots\<event_id>.png  <- page screenshot
└── verify_report.json          <- summary (ok/blocked/thin/title_mismatch counts)
```

Step 5 — Sentiment (score articles)

```
python sentiment.py
```

What it does: Sends each verified article's text to the LLM, gets a sentiment score and signal, aggregates per company.

Output:

```
output\sentiment_triage_<run_id>\
├── article_scores.json         <- per-article score, evidence quote, content hash
└── company_sentiment.json      <- per-company aggregate (net sentiment,
opportunity/threat counts)
```

Step 6 — Financial gate (prepare financials)

First, clean the CSVs if needed (removes rows with definition breaks):

```
cd ..\planning
python ..\clean_gm.py
```

Then run the gate:

```
python financial_gate.py --company "General Motors" `
  --balance "..\assets\GM_balance_sheet_annual_2020_2026_cleaned.csv" `
```

```
  --income "..\assets\GM_income_statement_annual_2020_2026_cleaned.csv" `
  --units thousands --currency USD --entity "consolidated group"
```

What it does: Maps each CSV row to a standard field, checks accounting identities, asks you to confirm ambiguous rows (equity, revenue), flags warnings, then freezes the dataset after your approval.

Interactive prompts you'll see:

- Equity: choose "Including Portion Attributable to Noncontrolling Interest" (option 2)
- Revenue: choose "Revenues" (option 1)
- Interest expense: leave unmapped (press Enter) if the row is incomplete
- Final approval: `y`

Output:

```
output\financial_<company>_<timestamp>\
├── gate_report.json            <- full mapping proposal + warnings (pre-approval)
├── approved_financials.json    <- frozen dataset (only after you approve)
├── mapping_approved.json       <- final row-to-field mapping
└── decisions.json              <- your choices recorded
```

Important: Only `approved_financials.json` is valid input for the SWOT builder. `gate_report.json` from a `--report-only` run is NOT approved.

Step 7 — SWOT build (combine everything)

```
cd ..\information_new
python swot_builder.py build `
  --sentiment output\sentiment_triage_<run_id> `
  --financial
"..\planning\output\financial_<company>_<timestamp>\approved_financials.json" `
  --objective "assess expansion readiness"
```

What it does:

- Loads verified news scores, groups them into stories
- Loads approved financial metrics, computes ratios/growth
- Sends items in batches of 8 to the LLM, which proposes a SWOT quadrant + qualified statement per item
- Validates every proposal with code rules (financial items can't be opportunity/threat, numbers must appear in evidence, etc.)
- Writes the draft

Output:

```
output\swot_<timestamp>\
├── swot_draft.json   <- full draft: every item, proposal, evidence, notes,
warnings
└── swot_draft.md     <- human-readable version grouped by quadrant
```

The JSON contains, per item: id, kind (news/financial), title, evidence (URL/hash/quote or file/metric), proposed quadrant, basis (fact/allegation/forecast/inference), materiality (1-3), statement, why, and any validation notes.

Step 8 — SWOT review (human approval)

```
python swot_builder.py review --draft output\swot_<timestamp>\swot_draft.json
```

What it does: Walks you through every proposal interactively.

Prompts per item:

- `[Enter]` = approve as proposed
- `e` = edit (change quadrant and/or statement)
- `r` = reject (with reason)
- `s` = skip (leave pending)
- `q` = quit early

Output:

```
output\swot_<timestamp>\
├── swot_final.json          <- only approved/edited items, grouped by quadrant
├── swot_final.md            <- clean final SWOT in markdown
└── review_decisions.json    <- log of every approve/edit/reject decision
```

Output files at a glance

Stage	Folder	Key files
Discovery	`information_new\output\<id>\`	`discovery_<id>.json`
Triage	`information_new\output\triage_<id>\`	`triage_<company>.json`, `selected_for_resolution.json`, `label_cache.json`
Resolve	same triage folder	`resolved.json`, `resolved_failures.json`
Verify	`information_new\output\verify_triage_<id>\`	`articles/*.json`, `raw/*.html`, `screenshots/*.png`,

Stage	Folder	Key files
		`verify_report.json`
Sentiment	`information_new\output\sentiment_triage_<id>\`	`article_scores.json`, `company_sentiment.json`
Financial gate	`planning\output\financial_<company>_<ts>\`	`gate_report.json`, `approved_financials.json`, `mapping_approved.json`, `decisions.json`
SWOT build	`information_new\output\swot_<ts>\`	`swot_draft.json`, `swot_draft.md`
SWOT review	same swot folder	`swot_final.json`, `swot_final.md`, `review_decisions.json`

Common pitfalls

- **401 Invalid API Key:** key is wrong, expired, or not set in this terminal. Re-set `$env:VULTR_INFERENCE_API_KEY` and `$env:GROQ_API_KEY`.
- **429 rate limit:** normal on free tier. The scripts wait and retry automatically. Don't Ctrl+C mid-run.
- **Keyword fallback in triage:** means the LLM call failed; labels are weaker. Re-run `triage.py` (cached labels are kept).
- **Passing `gate_report.json` instead of `approved_financials.json`:** the gate report is pre-approval; only the approved file is valid SWOT input.
- **Debt/interest rows removed in cleaned CSVs:** intentional, because the source changed definitions mid-series. Debt-to-equity and interest coverage will show as unavailable — that's correct, not a bug.
- **GM Financial caveat:** GM's consolidated statements include a captive finance arm, so leverage and liquidity ratios aren't comparable to pure manufacturers. Mention this in any final write-up.

Quick reference: full run for a new company

```
# 1. Set env vars (see Step 0)

# 2. News pipeline
cd information_new
python discovery.py --company "<Name>" --ticker <TICKER> --location global
python triage.py
python resolve.py
python verify.py
python sentiment.py

# 3. Financial gate
cd ..\planning
python ..\clean_<company>.py   # if needed
python financial_gate.py --company "<Name>" `
```

```
  --balance "..\assets\<company>_balance_sheet.csv" `
  --income "..\assets\<company>_income_statement.csv" `
  --units thousands --currency USD --entity "consolidated group"
# answer prompts, approve at the end

# 4. SWOT build + review
cd ..\information_new
python swot_builder.py build `
  --sentiment output\sentiment_triage_<id> `
  --financial
"..\planning\output\financial_<company>_<ts>\approved_financials.json" `
  --objective "assess expansion readiness"
python swot_builder.py review --draft output\swot_<ts>\swot_draft.json
```

That's the full loop. Every output is timestamped and hashed, so you can always trace which news run and which financial file produced a given SWOT draft.

RCA / 5 Whys (optional)

After the SWOT review, you can drill into any news-based weakness or threat:

```
python rca.py run --draft output\swot_<ts>\swot_draft.json `
  --item "story:<id>" `
  --note "your context note here"
```

Output: `rca_story_<id>.json` and `.md` in the same swot folder. Financial items may stop early (`stopped_insufficient_evidence`) since they lack article text — that's expected.